

Develop a text analysis agent

[Prerequisites](#)

[Create a Microsoft Foundry project](#)

[Create an agent](#)

[Create an Azure Language in Foundry Tools connection](#)

[Test the Azure Language tool in the playground](#)

[Configure tool approval](#)

[Create a client application](#)

[Test the client application](#)

[View tool details](#)

[Clean up resources](#)

Azure Language in Foundry Tools supports analysis of text, including language detection, entity recognition, and PII redaction.

You can use the service directly in an application through its REST API and several language-specific SDKs. You can also use the **Azure Language in Foundry Tools MCP server** to integrate its capabilities into an AI agent, which is what you'll do in this exercise.

The code used in this exercise is based on the Microsoft Foundry SDK for Python. You can develop similar solutions using the SDKs for Microsoft .NET, JavaScript, and Java. Refer to [Microsoft Foundry SDK client libraries](#) for details.

This exercise takes approximately **30** minutes.

Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](#)
- [Visual Studio Code](#) installed
- [Python version 3.13.xx](#) installed*
- [Git](#) installed and configured
- [Azure CLI](#) installed

* Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the [Microsoft Foundry portal](#) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the Foundry logo at the top left to navigate to the home page.
2. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
 - **Foundry resource:** *Use the default name for your resource (usually {project_name}-resource)*
 - **Subscription:** *Your Azure subscription*
 - **Resource group:** *Create or select a resource group*
 - **Region:** Select any available region

Note: Use a recommended Microsoft Foundry region. Model availability may vary by region.

TIP: Remember (or make a note of) the Foundry resource name - you're going to need it later!

3. Select **Create**. Wait for your project to be created.

4. On the home page for your project, note that the API key, project endpoint, and OpenAI endpoint are displayed here.

TIP: Copy the project key to the clipboard - you're going to need it later!

Create an agent

Now that you have a Foundry project, you can create an agent.

1. Now you're ready to **Start building**. Select **Create agents** (or on the **Build** page, select the **Agents** tab); and create a new agent named `Text-Analysis-Agent`.

When ready, your agent opens in the agent playground.

2. In the model drop-down list, ensure that a **gpt-5** model has been deployed and selected for your agent.

3. Assign your agent the following **Instructions**:

CodeCopy

You are an AI agent that assists users by helping them analyze text.

4. Use the **Save** button to save the changes.

5. Test the agent by entering the following prompt in the **Chat** pane:

CodeCopy

What can you help me with?

The agent should respond with an appropriate answer based on its instructions.

Create an Azure Language in Foundry Tools connection

Foundry includes an MCP server for Azure Language in Foundry Tools, which you can connect to your project and use in your agent.

1. In the navigation pane on the left, select the **Tools** page.
2. Within the Tools page select the **Tools** tab.
3. Connect a tool; selecting **Azure Language in Foundry Tools** in the **Catalog** and connecting it to an endpoint. specifying the following configuration

- **Name:** A unique name for your tool/
- **Remote MCP Server endpoint:**

https://{foundry-resource-name}.cognitiveservices.azure.com/language/mcp?api-version=2025-11-15-preview
- **Parameters:** foundry-resource-name: *Your foundry resource name*
- **Authentication:** Key-based:
- **Ocp-Apim-Subscription-Key:** *API Key for your Foundry project*

Note: If key-based authentication is disabled by a policy in your Azure subscription, you can use Entra ID authentication to connect the agent to the Azure Language service.

4. Wait for the MCP tool connection to be created, and then view its details page.

5. On the details page for the Azure Language in Foundry Tools connection, select **Use in an agent**, and then select the **Text-Analysis-Agent** agent you created previously.

The agent should open in the playground, with the Azure Language in Foundry Tools tool connected.

!

Note: In some cases, the MCP tool is not added to the agent automatically. Verify that **Azure Language in Foundry Tools** appears in the agent's tool list. If it does not, add it manually from the available tools for the agent.

Test the Azure Language tool in the playground

Now let's test the agent's ability to use the tool you connected.

1. In the agent playground for the **Text-Analysis-Agent** agent, modify the instructions as follows:

Code

Copy

```
You are an AI agent that assists users by helping them analyze text. Use the Azure Language tool to perform text analysis tasks.
```

2. Use the **Save** button to save the changes.
3. Test the agent by entering the following prompt in the **Chat** pane:

Code

Copy

```
Identify the PII entities in this article, and generate a redacted version:

Microsoft was founded on April 4, 1975, by childhood friends Bill Gates (then 19) and Paul Allen (22) after they were inspired by the Altair 8800, one of the first personal computers, featured on the cover of Popular Electronics. They contacted the Altair’s maker, MITS, and successfully developed a version of the BASIC programming language, despite initially not owning the machine themselves. The pair formed a partnership called “Micro-Soft” in Albuquerque, New Mexico, close to MITS’s headquarters, with the goal of writing software for emerging microcomputers.

In the late 1970s, Microsoft grew by supplying programming languages to multiple hardware vendors, then relocated to the Seattle area in 1979. A pivotal moment came in 1980 when Microsoft partnered with IBM to provide an operating system for the IBM PC, leading to MS-DOS and establishing the company’s dominance in personal computing. Gates guided the company’s long-term strategy as CEO, while Allen contributed key technical vision in its early years, setting Microsoft on a path that would reshape the software industry.
```

4. When prompted, approve use of the Azure Language tool by selecting **Always approve all Azure Language in Foundry Tools tools** (you may need to do this twice because the prompt asked for two distinct text analysis tasks).

!

Note: Depending on your environment, you may occasionally receive a 403, 502, or other transient error after approving the tool. These errors are usually not blocking. Refresh the page and retry the same prompt.

5. Review the response, which should identify any personally identifiable information in the article about the founding of Microsoft, and create a version of the article with this information redacted.
6. Review the **Logs** for the chat and verify that the Azure Language tool was used by the agent to process the prompt.

Configure tool approval

As you've seen in the playground, to use the tool, the agent needs approval.

1. In the playground, in the list of **Tools** under the **Instructions**, in the menu for the Azure language tool you added, select **Configure**.
2. Ensure that the **Approval setting for tools in this MCP server for this agent** setting is **Always auto-approve all tools** (if not, change it and add it).
3. Save any changes to the agent.

Create a client application

Now that you have a working agent, you can create a client application that uses it.

Get the application files from GitHub

1. Open Visual Studio Code.
2. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-language` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.
3. After the repo has been cloned, in the Explorer pane, navigate to the folder containing the application code files at **/Labfiles/02-language-agent/Python/text-agent**. The application files include:
 - **.env** (the application configuration file)
 - **requirements.txt** (the Python package dependencies that need to be installed)
 - **text-agent.py** (the code file for the application)

Configure the application

1. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
2. In the **Command Palette**, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.1x installation.

Tip: If you are prompted to install dependencies, you can install the ones in the *requirements.txt* file in the */Labfiles/02-language-agent/Python/text-agent* folder; but it's OK if you don't, we'll install them later.

Tip: If you prefer to use the terminal, you can create your **Venv** environment with `python -m venv labenv`, then activate it with `\labenv\Scripts\activate`.
3. In the **Explorer** pane, right-click the **text-agent** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the */Labfiles/02-language-agent/Python/text-agent* folder.)

Note: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.
4. Ensure that the terminal is open in the **text-agent** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.
5. Install the Foundry SDK package, the Azure Identity package, and other required packages by running the following command:

CodeCopy

```
pip install -r requirements.txt
```

6. In the **Explorer** pane, in the **text-agent** folder, select the **.env** file to open it. Then update the configuration values to include your project **endpoint** (from the project home page in Foundry Portal) and the name of your agent (which should be **Text-Analysis-Agent** - note that this name is case-sensitive).
7. Save the modified configuration file.

Implement application code

1. In the **Explorer** pane, in the **text-agent** folder, open the **text-agent.py** file.
2. Review the existing code. You will add code to submit prompts to your agent.

 **Tip:** As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespaces you will need:

CodeCopy

```
# import namespaces
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
```

4. In the **main** function, note that code to load the endpoint from the configuration file has already been provided. Then find the comment **Get project client**, and add the following code to create a client for your Foundry project:

CodeCopy

```
# Get project client
project_client = AIProjectClient(
    endpoint=foundation_endpoint,
    credential=DefaultAzureCredential(),
)
```

5. Find the comment **Get an OpenAI client**, and add the following code to get an OpenAI client with which to call your agent.

CodeCopy

```
# Get an OpenAI client
openai_client = project_client.get_openai_client()
```

6. Find the comment **Use the agent to get a response**, and add the following code to submit a user prompt to your agent, and display the response.

CodeCopy

```
# Use the agent to get a response
prompt = input("User prompt: ")
response = openai_client.responses.create(
    input=[{"role": "user", "content": prompt}],
    extra_body={"agent_reference": {"name": agent_name, "type": "agent_reference"}},
)

print(f"{agent_name}: {response.output_text}")
```

7. Save the changes you made to the code file.

Test the client application

Now let's test the application by running it in a Python environment and authenticating the connection to your project.

1. In the Visual Studio Code terminal, enter the following command to sign into Azure

Code

 Copy

```
az login
```

!

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See [Sign into Azure interactively using the Azure CLI](#) for details.

2. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

3. After you have signed in, enter the following command to run the application:

Code

 Copy

```
python text-agent.py
```

4. When prompted, enter the following prompt:

Code

 Copy

```
Extract named entities from the following text: "Pierre and I went to Paris on July 14th."
```

5. Review the response, which should identify named people, places, and dates.

View tool details

The Azure Language in Foundry Tools tool provides a wide range of functionality, and the agent must select the appropriate function to call. We can see the options available in the agent's response.

1. In the **text-agent.py** code file, add the following line immediately after the *print(f"{agent_name}: {response.output_text}")* line you added previously (before the *except Exception as ex:* line):

Code

 Copy

```
print(f"\nResponse Details: {response.model_dump_json(indent=2)}")
```

2. Save the changes to the code file.

3. In the terminal, re-enter the command to run the application (`python text-agent.py`).

4. When prompted, enter the following command:

Code

 Copy

Tell me what entities **and** dates are mentioned **in this** review, **and** whether it **is** positive **or** negative: "I booked my flight to Paris in July with Margie's Travel, and it was fantastic!"

5. Review the response (you may need to scroll quite far up to see it), which should identify entities and dates, and determine the sentiment of the text.
6. Review the JSON response details, which indicate each of the tools available to the agent. In this case, it should have used the **extract_named_entities_from_text** and **detect_sentiment_from_text** tools within Azure Language in Foundry Tools.

Clean up resources

If you're finished exploring the Azure Language service, you can delete the resources you created in this exercise. Here's how:

1. In the Azure portal, browse to the Foundry resource you created in this lab.
2. On the resource page, select **Delete** and follow the instructions to delete the resource.